

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA0614	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	Mohan Kumar J	Reg. No.:	192424060
Section / Batch:	AI & DS	Date:	

Instructions to Students

- Answer BOTH scenario-based questions. Each question carries 50 Marks. Total: 100 Marks.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions	Marks
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	

SECTION A – SCENARIO BASED ASSESSMENT

Question 1:

A hospital database stores patient records unsorted and retrieves them using Linear Search during emergencies.

- Explain how Linear Search operates.
- Analyze its efficiency.
- Justify whether sorting is beneficial.

Question 2:

An inventory system searches for item IDs in an unsorted dataset using Linear Search.

- Explain how Linear Search is applied.
- Analyze performance degradation with large data.
- Suggest improvements and justify.

Code of Conduct

I Mohan Kumar J (I92424060) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Mohan Kumar J

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Question No.	Problem Understanding & Context Analysis (25%)	Application of Concepts (30%)	Analytical & Decision-Making Skills (20%)	Solution Design & Approach (15%)	Clarity, Justification & Presentation (10%)	Total Marks (Each – 50)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1

Hospital Database Using Linear Search During Emergencies

Introduction

In a hospital management system, patient records are often stored in a database containing information such as Patient ID, Name, Age, Medical History, Diagnosis, and Emergency Contact. When the records are stored in an **unsorted** manner, locating a patient's information requires the system to examine records sequentially. Linear Search is the most suitable searching technique in such situations because it does not require the data to be sorted beforehand.

a) Explain how Linear Search operates?

Linear Search is a sequential searching algorithm that examines each record one by one until the required patient record is found or the dataset ends.

Working Procedure

1. Start searching from the first patient record.
2. Compare the entered Patient ID with the current record.
3. If both IDs match, return the patient details immediately.
4. Otherwise, move to the next record.
5. Continue the process until:
 - The patient record is found, or
 - All records have been checked.

Algorithm

```
LinearSearch(PatientRecords[], key)
{
    for(i = 0; i < n; i++)
    {
        if(PatientRecords[i] == key)
            return i;
    }
    return -1;
}
```

Example

Patient IDs:

[105, 210, 156, 341, 450, 198]

Search Patient ID = **341**

Step	Compared ID	Result
1	105	Not Found
2	210	Not Found
3	156	Not Found
4	341	Found

The patient record is found after **4 comparisons**.

b) Analyze its efficiency.

The efficiency of Linear Search depends on the position of the patient record.

Time Complexity

Case	Complexity	Explanation
Best Case	$O(1)$	Record found in the first comparison
Average Case	$O(n)$	Record found in the middle
Worst Case	$O(n)$	Record found at the last position or absent

Space Complexity

$O(1)$

Only a few variables are used regardless of database size.

Efficiency Analysis

Suppose there are 100,000 patient records.

- Best Case → 1 comparison
- Average Case → Approximately 50,000 comparisons
- Worst Case → 100,000 comparisons

During emergencies, large numbers of comparisons increase retrieval time and may delay access to critical patient information. Therefore, Linear Search becomes inefficient for very large hospital databases.

c) Justify whether sorting is beneficial?

Sorting the database is beneficial when searches are performed frequently.

Advantages of Sorting

- Enables Binary Search.
- Reduces search time significantly.
- Improves emergency response.
- Faster retrieval for repeated searches.
- Better database organization.

Comparison

Linear Search	Binary Search
Works on unsorted data	Requires sorted data
Time Complexity = $O(n)$	Time Complexity = $O(\log n)$
Simple implementation	Much faster for large datasets

Decision and Justification

Although sorting requires additional preprocessing time ($O(n \log n)$), hospital databases are searched much more frequently than they are updated. Therefore, maintaining records in sorted order is highly beneficial because Binary Search can retrieve patient information much faster during emergencies.

For dynamic databases with frequent insertions, balanced trees or hash tables can also be considered to achieve efficient searching without repeated sorting.

Conclusion

Linear Search is simple and suitable for unsorted hospital databases. However, its linear time complexity causes performance degradation as the number of patient records increases. Sorting the database enables Binary Search, reducing search time from $O(n)$ to $O(\log n)$, making it the preferred solution for emergency patient retrieval systems.

main.py

Share

Run

```
--
15 # Sample Hospital Database
16 patients = [
17     {"Patient ID": 101, "Name": "Rahul", "Age": 25},
18     {"Patient ID": 205, "Name": "Priya", "Age": 32},
19     {"Patient ID": 310, "Name": "Karan", "Age": 41},
20     {"Patient ID": 450, "Name": "Anjali", "Age": 28},
21     {"Patient ID": 512, "Name": "Ravi", "Age": 36}
22 ]
23
24 search_id = int(input("Enter Patient ID to search: "))
25
26 position, comparisons = linear_search(patients, search_id)
27
28 if position != -1:
29     print("\nPatient Found")
30     print("Patient ID :", patients[position]["Patient ID"])
31     print("Name      :", patients[position]["Name"])
32     print("Age       :", patients[position]["Age"])
33 else:
34     print("\nPatient Not Found")
35
36 print("Comparisons Made :", comparisons)
37 print("Time Complexity : O(n)")
38 print("Space Complexity: O(1)")
```

Output

Enter Patient ID to search: 450

Patient Found

Patient ID : 450

Name : Anjali

Age : 28

Comparisons Made : 4

Time Complexity : O(n)

Space Complexity: O(1)

=== Code Execution Successful ===

Question 2:

Inventory System Using Linear Search

Introduction

An inventory management system stores thousands of item IDs representing products in warehouses. When the dataset is unsorted, the system uses Linear Search to locate a requested item by comparing each item ID sequentially. Although easy to implement, its performance decreases as inventory size increases.

a) Explain how Linear Search is applied.

Linear Search scans each item ID until the required item is found.

Working Procedure

1. Begin with the first item.
2. Compare the entered Item ID with the current record.
3. If matched, return the item details.
4. Otherwise, continue searching.
5. Stop when the item is found or all records are checked.

Algorithm

```
LinearSearch(ItemID[], key)
{
    for(i = 0; i < n; i++)
    {
        if(ItemID[i] == key)
            return i;
    }
    return -1;
}
```

Example

```
Inventory IDs
[320, 145, 678, 456, 901, 234]
Search Item ID = 456
```

Step	Compared ID	Result
------	-------------	--------

1	320	Not Found
2	145	Not Found
3	678	Not Found
4	456	Found

The required item is located after **4 comparisons**.

b) Analyze performance degradation with large data.

As inventory size grows, Linear Search must inspect more records before finding the desired item.

Time Complexity

Case	Complexity
Best Case	$O(1)$
Average Case	$O(n)$
Worst Case	$O(n)$

Space Complexity

$O(1)$

Performance Analysis

Assume an inventory containing **1,000,000 items**.

- Best Case → 1 comparison
- Average Case → Approximately 500,000 comparisons
- Worst Case → 1,000,000 comparisons

Impact

- Increased response time.
- Reduced warehouse efficiency.
- Higher CPU utilization.
- Slower billing and stock updates.
- Poor scalability for enterprise inventory systems.

Hence, Linear Search becomes unsuitable for very large inventories.

c) Suggest improvements and justify.

Several techniques can improve search performance.

1. Sort the Dataset and Use Binary Search

- Time Complexity = **$O(\log n)$**
- Suitable when searches are frequent.

2. Use Hash Tables

- Average Search Complexity = **$O(1)$**
- Excellent for searching item IDs.
- Commonly used in inventory databases.

3. Use Balanced Search Trees

Examples:

- AVL Tree
- Red-Black Tree

Search Complexity:

$O(\log n)$

Efficient for datasets with frequent insertions and deletions.

Comparison

Searching Method	Time Complexity	Suitable Scenario
Linear Search	$O(n)$	Small unsorted datasets
Binary Search	$O(\log n)$	Large sorted datasets
Hash Table	$O(1)$ Average	Fast inventory lookup
AVL / Red-Black Tree	$O(\log n)$	Dynamic inventory systems

Justification

For modern inventory systems, Hash Tables provide the best performance because item IDs are unique and can be accessed directly without sequential comparisons. If maintaining sorted data is required, **Binary Search** offers an efficient alternative. These improvements significantly reduce search time, improve scalability, and enhance the overall efficiency of the inventory management system.

Conclusion

Linear Search is appropriate for small and unsorted inventories due to its simplicity. However, its **$O(n)$** time complexity leads to poor performance as the dataset grows. Implementing Hash Tables or Binary Search on sorted data provides much faster retrieval, improves scalability, minimizes search delays, and ensures efficient inventory management in large-scale applications.

main.py



Share

Run

Output

```
14
15 # Sample Inventory
16 inventory = [
17     {"Item ID": 1201, "Item Name": "Keyboard"},
18     {"Item ID": 1202, "Item Name": "Mouse"},
19     {"Item ID": 1203, "Item Name": "Monitor"},
20     {"Item ID": 1204, "Item Name": "Printer"},
21     {"Item ID": 1205, "Item Name": "Scanner"}
22 ]
23
24 search_item = int(input("Enter Item ID to search: "))
25
26 position, comparisons = linear_search(inventory, search_item)
27
28 if position != -1:
29     print("\nItem Found")
30     print("Item ID :", inventory[position]["Item ID"])
31     print("Item Name :", inventory[position]["Item Name"])
32 else:
33     print("\nItem Not Found")
34
35 print("Comparisons Made :", comparisons)
36 print("Time Complexity : O(n)")
37 print("Space Complexity: O(1)")
```

Enter Item ID to search: 1204

Item Found

Item ID : 1204

Item Name : Printer

Comparisons Made : 4

Time Complexity : O(n)

Space Complexity: O(1)

=== Code Execution Successful ===